2.

a. Chatgpt and Gemini
b. I accessed them through browser
c.

Write a C++ program to do this prompt:

Sudoku is a number puzzle in which a 9x9 grid is subdivided into 9 3x3 sub-grids, or blocks.
• Each of the cells in the puzzle must be filled in with a single digit 1-9, subject to the following constraints:
o Each digit from 1 to 9 must appear exactly once in each row.
o Each digit from 1 to 9 must appear exactly once in each column.
o Each digit from 1 to 9 must appear exactly once in each block.
• Published puzzles are usually constructed to have a unique solution. This requires that at least 17 cells be filled in (and may require more; depending on the puzzle and which cells are empty, as many as 78 filled cells can be required to guarantee uniqueness). In general, the number of cells that are already filled in is not a particularly good indicator of how difficult a Sudoku puzzle is to solve.
• The Sudoku problem, then, is this: Given a partially completed Sudoku puzzle, find the (or at least a) solution; if no solution exists, prove that.
• It is possible to solve a Sudoku puzzle entirely through logic. For example, in the above puzzle, consider the bottom right block, top right cell. Because of other numbers in the same row, column, or grid, that number cannot be 1, 2, 3, 5, 6, 7, 8, or 9, and so must be 4. By proceeding in this fashion, using these and more advanced deduction rules, it’s possible to gradually fill in the entire grid.
• That is not the approach we’re going to use. Instead, we’re going to implement a brute-force search, in which we exhaustively try possibilities until we find a solution. This will be slower to run but is easier to solve. (And by “slower to run,” we mean it'll finish in a few tenths of a second rather than a few milliseconds.)
• Here’s the idea: Suppose we have a Sudoku puzzle of which k of the 81 cells are unfilled. We identify an empty cell and deduce a list of candidates for what might go into that cell. If there are no such values (all digits are accounted for elsewhere in the same row, column, or grid), then we must have made a mistake in filling in the k cells and return a no-solution-found result. Otherwise, we try putting the first candidate value into the empty cell, and see if we can find a solution with the k+1 cells filled (i.e. a recursive call). If we get a no-solution-found to the first candidate, then we try the next, and so on, until we either find a solution (k == 81) or determine that no solution exists (all candidates report no solution, which again implies we took a wrong path somewhere earlier).

• This approach, by the way, is called depth-first search with backtracking. For each empty cell we make an attempt and see how far we can get with it. If we fail to find a solution, we back up until we find something else to try, and so on. If a solution exists, we are guaranteed to find it, because we exhaustively try all possibilities.
• You are given five files (puzzle1.txt through puzzle5.txt), each containing a Sudoku puzzle. Digits already filled in have been placed, and an underscore character (_) is used to indicate a blank square.
• Write a recursive object-oriented program to find the solution to the five puzzle files.
• The output for each solution, should include:
o The puzzle file name (e.g., puzzle1.txt)
o The puzzle from the file printed out.
o The solution to the puzzle printed out or if no solution is found, “No solution found” printed out.
• The solution may not be unique; if there is more than one solution to the puzzle, your program should print them all out.

d.

Chatgpt

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

class Sudoku {
private:
    int grid[9][9];
    int original[9][9];
    int solutionCount;

    // Check whether a number can be placed in a cell.
    bool isSafe(int row, int col, int num) {
        // Check the row.
        for (int c = 0; c < 9; c++) {
            if (grid[row][c] == num) {
                return false;
            }
        }

        // Check the column.
```

```
    for (int r = 0; r < 9; r++) {
        if (grid[r][col] == num) {
            return false;
        }
    }

    // Find the top-left corner of the 3x3 block.
    int startRow = (row / 3) * 3;
    int startCol = (col / 3) * 3;

    // Check the 3x3 block.
    for (int r = startRow; r < startRow + 3; r++) {
        for (int c = startCol; c < startCol + 3; c++) {
            if (grid[r][c] == num) {
                return false;
            }
        }
    }

    return true;
}

// Find the next empty cell.
bool findEmptyCell(int &row, int &col) {
    for (row = 0; row < 9; row++) {
        for (col = 0; col < 9; col++) {
            if (grid[row][col] == 0) {
                return true;
            }
        }
    }

    return false;
}

// Print the current Sudoku grid.
void printGrid() {
    for (int row = 0; row < 9; row++) {
        if (row % 3 == 0 && row != 0) {
            cout << "------+-------+------" << endl;
        }

        for (int col = 0; col < 9; col++) {
            if (col % 3 == 0 && col != 0) {
```

```
            cout << "| ";
        }

        if (grid[row][col] == 0) {
            cout << "_ ";
        } else {
            cout << grid[row][col] << " ";
        }
    }

    cout << endl;
  }
}

// Check whether the original puzzle follows the rules.
bool isValidPuzzle() {
  for (int row = 0; row < 9; row++) {
    for (int col = 0; col < 9; col++) {
      int num = grid[row][col];

      if (num != 0) {
        // Temporarily remove the number so it
        // does not conflict with itself.
        grid[row][col] = 0;

        bool valid = isSafe(row, col, num);

        grid[row][col] = num;

        if (!valid) {
          return false;
        }
      }
    }
  }

  return true;
}

// Recursively find and print every solution.
void solveAll() {
  int row, col;

  // If there are no empty cells, a solution is found.
```

```
        if (!findEmptyCell(row, col)) {
            solutionCount++;

            cout << "\nSolution " << solutionCount << ":" << endl;
            printGrid();
            return;
        }

        // Try every possible number in the empty cell.
        for (int num = 1; num <= 9; num++) {
            if (isSafe(row, col, num)) {
                // Place the number.
                grid[row][col] = num;

                // Recursively solve the remaining puzzle.
                solveAll();

                // Backtrack and try another number.
                grid[row][col] = 0;
            }
        }
    }

public:
    Sudoku() {
        solutionCount = 0;

        for (int row = 0; row < 9; row++) {
            for (int col = 0; col < 9; col++) {
                grid[row][col] = 0;
                original[row][col] = 0;
            }
        }
    }

    // Read a Sudoku puzzle from a file.
    bool loadPuzzle(string filename) {
        ifstream inputFile(filename);

        if (!inputFile) {
            cout << "Error opening " << filename << endl;
            return false;
        }
```

```
    int count = 0;
    char ch;

    // Read digits and underscores, ignoring whitespace.
    while (inputFile >> ch) {
        if (ch >= '1' && ch <= '9') {
            if (count >= 81) {
                cout << "Invalid puzzle file: "
                     << filename << endl;
                return false;
            }

            grid[count / 9][count % 9] = ch - '0';
            count++;
        } else if (ch == '_') {
            if (count >= 81) {
                cout << "Invalid puzzle file: "
                     << filename << endl;
                return false;
            }

            grid[count / 9][count % 9] = 0;
            count++;
        }
    }

    inputFile.close();

    if (count != 81) {
        cout << "Invalid puzzle file: " << filename
             << " must contain exactly 81 cells." << endl;
        return false;
    }

    // Save the original puzzle for printing.
    for (int row = 0; row < 9; row++) {
        for (int col = 0; col < 9; col++) {
            original[row][col] = grid[row][col];
        }
    }

    return true;
}
```

```
    // Print the original puzzle.
    void printOriginal() {
        for (int row = 0; row < 9; row++) {
            if (row % 3 == 0 && row != 0) {
                cout << "------+-------+------" << endl;
            }

            for (int col = 0; col < 9; col++) {
                if (col % 3 == 0 && col != 0) {
                    cout << "| ";
                }

                if (original[row][col] == 0) {
                    cout << "_ ";
                } else {
                    cout << original[row][col] << " ";
                }
            }

            cout << endl;
        }
    }

    // Solve the puzzle and print every solution.
    void solve() {
        solutionCount = 0;

        if (!isValidPuzzle()) {
            cout << "\nNo solution found." << endl;
            return;
        }

        solveAll();

        if (solutionCount == 0) {
            cout << "\nNo solution found." << endl;
        } else {
            cout << "\nTotal solutions found: "
                 << solutionCount << endl;
        }
    }
};

int main() {
```

```
    string filenames[5] = {
        "puzzle1.txt",
        "puzzle2.txt",
        "puzzle3.txt",
        "puzzle4.txt",
        "puzzle5.txt"
    };

    for (int i = 0; i < 5; i++) {
        cout << "\n================================" << endl;
        cout << "File: " << filenames[i] << endl;
        cout << "================================" << endl;

        Sudoku puzzle;

        if (puzzle.loadPuzzle(filenames[i])) {
            cout << "\nOriginal Puzzle:" << endl;
            puzzle.printOriginal();

            cout << "\nSolving..." << endl;
            puzzle.solve();
        }

        cout << endl;
    }

    return 0;
}
```

Gemini

```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>

using namespace std;

class Sudoku {
private:
    vector<vector<char>> board;
```

```
// Checks if placing 'num' at board[row][col] is valid
bool isValid(int row, int col, char num) const {
    for (int i = 0; i < 9; i++) {
        // Check row and column
        if (board[row][i] == num) return false;
        if (board[i][col] == num) return false;
    }

    // Check 3x3 sub-grid (block)
    int startRow = row - row % 3;
    int startCol = col - col % 3;
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            if (board[startRow + i][startCol + j] == num) return false;
        }
    }
    return true;
}

// Recursive backtracking function to find all solutions
void solveAll(int row, int col, int& solutionCount) {
    // Reached the end of the board: a valid solution is complete
    if (row == 9) {
        solutionCount++;
        cout << "Solution " << solutionCount << ":\n";
        printBoard();
        cout << "--------------------\n";
        return;
    }

    // Determine coordinates for the next cell
    int nextRow = (col == 8) ? row + 1 : row;
    int nextCol = (col == 8) ? 0 : col + 1;

    // If the current cell is already filled, skip to the next cell
    if (board[row][col] != '_') {
        solveAll(nextRow, nextCol, solutionCount);
        return;
    }

    // Try digits '1' through '9'
    for (char num = '1'; num <= '9'; num++) {
        if (isValid(row, col, num)) {
            board[row][col] = num; // Place candidate
```

```
            solveAll(nextRow, nextCol, solutionCount); // Recurse
            board[row][col] = '_'; // Backtrack
        }
    }
}

public:
    Sudoku() {
        board.resize(9, vector<char>(9, '_'));
    }

    // Load puzzle configuration from a text file
    bool loadFromFile(const string& filename) {
        ifstream file(filename);
        if (!file.is_open()) {
            return false;
        }

        for (int i = 0; i < 9; i++) {
            for (int j = 0; j < 9; j++) {
                if (!(file >> board[i][j])) {
                    return false;
                }
            }
        }
        return true;
    }

    // Print the Sudoku board
    void printBoard() const {
        for (int i = 0; i < 9; i++) {
            for (int j = 0; j < 9; j++) {
                cout << board[i][j] << " ";
            }
            cout << "\n";
        }
    }

    // Process and solve a specific puzzle file
    void solvePuzzle(const string& filename) {
        cout << "Puzzle File: " << filename << "\n";
        if (!loadFromFile(filename)) {
            cout << "Error: Could not open or read file " << filename << "\n\n";
            return;
```

```
        }

        cout << "Original Puzzle:\n";
        printBoard();
        cout << "-------------------\n";

        int solutionCount = 0;
        solveAll(0, 0, solutionCount);

        if (solutionCount == 0) {
            cout << "No solution found.\n";
        } else {
            cout << "Total solutions found: " << solutionCount << "\n";
        }
        cout << "========================================\n\n";
    }
};

int main() {
    Sudoku solver;

    // Process puzzle1.txt through puzzle5.txt
    for (int i = 1; i <= 5; i++) {
        string filename = "puzzle" + to_string(i) + ".txt";
        solver.solvePuzzle(filename);
    }

    return 0;
}
```

e.

	chatgpt	gemini
Correctness	Gave the right answer on all five tests. Rejects puzzles whose given digits conflict, so it printed “No solution found.” for the duplicate-5 puzzle in 0.004 s. Also validates characters and requires exactly 81 cells. Prints block separators.	Right answer on the four valid tests, but no validity check on the givens. On the duplicate-5 puzzle it did not finish within 120 s, because it kept enumerating invalid “solutions”. Accepts any character, and a failed read can leave a half-loaded board.

Execution Time	Overall: O(9^k), where k is the number of blank cells. Per recursive call: O(81 + 9·27) = O(1). findEmptyCell() rescans the grid from cell (0,0) on every call, which is up to 81 checks. isSafe() then scans the row, column, and block (27 checks) for up to 9 digits. One-time setup: O(81·27) = O(1). isValidPuzzle() stops the search right away when the given digits conflict.	Overall: O(9^k), where k is the number of blank cells. Per recursive call: O(9·27) = O(1). The cell index moves forward by one, so there is no rescan. isValid() scans the row, column, and block (27 checks) for up to 9 digits. One-time setup: none. With conflicting givens, it searches a tree that cannot contain a valid solution.
Space Complexity	O(1) beyond recursion depth (at most 81 frames). Fixed int[9][9] arrays, plus an extra copy of the puzzle.	O(1) beyond recursion depth (at most 81 frames). Heap-allocated nested vectors, with no copy of the original.
Maintainability	Clear function names and comments. printGrid() and printOriginal() duplicate each other. Magic numbers (9, 3, 81) throughout.	Shorter, with one reused print function and const methods. Mixes file reading, solving, and printing inside solvePuzzle(). No input validation.

f. I picked ChatGPT because correctness comes first, and its weaknesses (rescanning, duplicate printing) are easier to fix than Gemini's missing validation.

g.

- Correctness:
  - Kept the check for conflicting givens.
  - Added rejection of invalid characters and files with too many cells.
  - Added clear() so one object can be reused for all five puzzles.
  - Switched the solution counter to long long, since a sparse puzzle can have a very large number of solutions.
- Execution time:
  - Replaced the O(27) isSafe scan with row, column, and block bitmasks. Legal candidates are now one bit operation: ALL_DIGITS & ~(row | col | block).
  - Replaced the full-grid blank search with a position index passed down the recursion, adopted from Gemini.
- Space:
  - Used one flat 81-int array plus 27 mask integers, with no heap allocation.

    - The recursion depth is unchanged at 81 or fewer.
- Maintainability:
    - Merged the two print functions into one printBoard(const int[]).
    - Added named constants (SIZE, CELLS, ALL_DIGITS) instead of magic numbers.
    - Added const-correctness and a helper setUsed() so place and backtrack stay symmetric.